

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
- GitHub Repository Link in GCR
- Google Colab Share Link in GCR
- Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject-verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

Experiment Description

Design a Python program to validate whether a sentence follows the given Context-Free Grammar (CFG) rules and generate its parse tree representation.

Grammar:

$S \rightarrow NP VP$
 $NP \rightarrow Det N$
 $VP \rightarrow V NP$
 $Det \rightarrow the \mid a$
 $N \rightarrow dog \mid cat \mid bone$
 $V \rightarrow chased \mid found$

Task

1. Accept a sentence as input.
2. Verify whether it conforms to the CFG.
3. Construct and return the parse tree.
4. Return "Invalid Sentence" if parsing fails.

Function Prototype

```
def parse_cfg(sentence):  
    pass
```

Test Cases

Input	Expected Output
the dog chased a cat	Valid Parse Tree
a cat found the bone	Valid Parse Tree
dog chased cat	Invalid Sentence
the bone found a dog	Valid Parse Tree
chased the dog bone	Invalid Sentence

```
CO3 AT1 1.py - C:/Users/Admin/OneDrive/Documents/NLP/lab/CO3 AT1 1.py (3.12.5)  
File Edit Format Run Options Window Help  
def parse_cfg(sentence):  
    words = sentence.lower().split()  
    if len(words) != 5:  
        return "Invalid Sentence"  
    determiners = {"the", "a"}  
    nouns = {"dog", "cat", "bone"}  
    verbs = {"chased", "found"}  
    if words[0] not in determiners:  
        return "Invalid Sentence"  
    if words[1] not in nouns:  
        return "Invalid Sentence"  
    if words[2] not in verbs:  
        return "Invalid Sentence"  
    if words[3] not in determiners:  
        return "Invalid Sentence"  
    if words[4] not in nouns:  
        return "Invalid Sentence"  
    parse_tree = (  
        "S\n"  
        "├── NP\n"  
        "│   ├── Det → {words[0]}\n"  
        "│   └── N → {words[1]}\n"  
        "└── VP\n"  
        "    ├── V → {words[2]}\n"  
        "    └── NP\n"  
        "        ├── Det → {words[3]}\n"  
        "        └── N → {words[4]}\n"  
    )  
    return parse_tree  
test_sentences = [  
    "the dog chased a cat",  
    "a cat found the bone",  
    "dog chased cat",  
    "the bone found a dog",  
    "chased the dog bone"  
]  
print("EXPERIMENT 1: CFG PARSING")  
print("=" * 50)  
for sentence in test_sentences:  
    print("\nInput:", sentence)  
    result = parse_cfg(sentence)  
    print("Output:")  
    print(result)
```

EXPERIMENT 1: CFG PARSING
=====

Input: the dog chased a cat
Output:
S
├── NP
│ ├── Det → the
│ └── N → dog
└── VP
 ├── V → chased
 └── NP
 ├── Det → a
 └── N → cat

Input: a cat found the bone
Output:
S
├── NP
│ ├── Det → a
│ └── N → cat
└── VP
 ├── V → found
 └── NP
 ├── Det → the
 └── N → bone

Input: dog chased cat
Output:
Invalid Sentence

Input: the bone found a dog
Output:
S
├── NP
│ ├── Det → the
│ └── N → bone
└── VP
 ├── V → found
 └── NP
 ├── Det → a
 └── N → dog

Input: chased the dog bone
Output:
Invalid Sentence

Experiment 2: Agreement and Feature Structure Checking

Experiment Description

Design a Python program that verifies subject–verb agreement using feature structures containing Number and Person attributes.

Feature Rules

he → singular, third person
she → singular, third person
it → singular, third person
we → plural

eats → singular verb
sleeps → singular verb

eat → plural verb
sleep → plural verb

Task

1. Accept Subject and Verb.
2. Compare feature structures.
3. Return True if agreement holds.
4. Return False otherwise.

Function Prototype

```
def check_agreement(subject, verb):  
    pass
```

Test Cases

Subject Verb Expected

he	eats	True
he	eat	False
we	eat	True
we	sleep	False
she	sleep	True

```
CO3 AT1 2.py - C:/Users/Admin/OneDrive/Documents/NLP/lab/CO3 AT1 2.py (3.12.5)  
File Edit Format Run Options Window Help  
subject_features = {  
    "he": {  
        "number": "singular",  
        "person": "third"  
    },  
    "she": {  
        "number": "singular",  
        "person": "third"  
    },  
    "it": {  
        "number": "singular",  
        "person": "third"  
    },  
    "we": {  
        "number": "plural",  
        "person": "first"  
    }  
}  
verb_features = {  
    "eats": {  
        "number": "singular"  
    },  
    "sleeps": {  
        "number": "singular"  
    },  
    "eat": {  
        "number": "plural"  
    },  
    "sleep": {  
        "number": "plural"  
    }  
}  
def check_agreement(subject, verb):  
    subject = subject.lower()  
    verb = verb.lower()  
    if subject not in subject_features:  
        return False  
    if verb not in verb_features:  
        return False  
    subject_info = subject_features[subject]  
    verb_info = verb_features[verb]  
    if subject_info["number"] == verb_info["number"]:  
        return True  
    return False  
print("EXPERIMENT 2: FEATURE STRUCTURES")  
.....  
Ln: 1 Col: 15  
Ln: 28 Col: 0  
IDLE Shell 3.12.5  
File Edit Shell Debug Options Window Help  
Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug 6 2024, 20:45:27) [MSC v.1940 64 bit (AMD64)]  
on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>>  
= RESTART: C:/Users/Admin/OneDrive/Documents/NLP/lab/CO3 AT1 2.py  
EXPERIMENT 2: FEATURE STRUCTURES  
Subject Features:  
he : {'number': 'singular', 'person': 'third'}  
she : {'number': 'singular', 'person': 'third'}  
it : {'number': 'singular', 'person': 'third'}  
we : {'number': 'plural', 'person': 'first'}  
Verb Features:  
eats : {'number': 'singular'}  
sleeps : {'number': 'singular'}  
eat : {'number': 'plural'}  
sleep : {'number': 'plural'}  
AGREEMENT TEST CASES  
he + eats -> True  
he + eat -> False  
we + eat -> True  
we + sleeps -> False  
she + sleeps -> True  
>>>
```

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

Experiment Description

Implement a Python program to compute the probability of a sentence using the given PCFG.

Grammar

$S \rightarrow NP \ VP \quad [1.0]$
 $NP \rightarrow Tom \quad [0.7]$
 $NP \rightarrow Ann \quad [0.3]$
 $VP \rightarrow sings \quad [0.4]$
 $VP \rightarrow dances \quad [0.6]$

Probability Formula

$P(\text{Sentence})$
 $=$
 $P(S \rightarrow NP \ VP)$
 $\times P(NP)$
 $\times P(VP)$

Task

1. Accept a two-word sentence.
2. Parse using the PCFG.
3. Compute sentence probability.
4. Return 0 if sentence cannot be generated.

Function Prototype

```
def sentence_probability(sentence):  
    pass
```

Test Cases

Input Sentence Expected Probability

Tom sings	0.28
Tom dances	0.42
Ann sings	0.12
Ann dances	0.18
Sam sings	0.00

```
grammar = {
    "S": {
        ("NP", "VP"): 1.0
    },
    "NP": {
        ("Tom",): 0.7,
        ("Ann",): 0.3
    },
    "VP": {
        ("sings",): 0.4,
        ("dances",): 0.6
    }
}

def sentence_probability(sentence):
    words = sentence.split()
    if len(words) != 2:
        return 0.0

    subject = words[0]
    verb = words[1]
    if (subject,) not in grammar["NP"]:
        return 0.0
    if (verb,) not in grammar["VP"]:
        return 0.0
    p_s = grammar["S"][("NP", "VP")]
    p_np = grammar["NP"][(subject,)]
    p_vp = grammar["VP"][(verb,)]
    probability = p_s * p_np * p_vp
    return probability

test_sentences = [
    "Tom sings",
    "Tom dances",
    "Ann sings",
    "Ann dances",
    "Sam sings"
]

print("EXPERIMENT 3: PCFG SENTENCE PROBABILITY")
print("-" * 50)
for sentence in test_sentences:
    probability = sentence_probability(sentence)
    print(f"Input: {sentence}")
    print(f"Probability: {probability:.2f}")
    print("-" * 50)
```

```
Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug 6 2024, 20:45:27) [MSC v.1940 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
= RESTART: C:/Users/Admin/OneDrive/Documents/NLP/lab/CO3 AT1 3.py
EXPERIMENT 3: PCFG SENTENCE PROBABILITY
=====
Input: Tom sings
Probability: 0.28
-----
Input: Tom dances
Probability: 0.42
-----
Input: Ann sings
Probability: 0.12
-----
Input: Ann dances
Probability: 0.18
-----
Input: Sam sings
Probability: 0.00
=====
```

>>>

ubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____